```
                                    none_perm = sorted(list(all_u - (read_s | write_s | del_s)))
```

PYTHON PLACEMENT PRACTICE: 100 LEVEL 4

Complete Implementation, Source Code & Terminal Execution Verification

Q001 | Ranked transaction totals

Source Implementation:

```
def ranked_transactions(amounts):



                best = {}
```

Verified Output:

```
[('8.50', 2), ('12.34', 1), ('12.35', 1), ('3.00', 1)]
                            all_three = sorted(list(read_s & write_s & del_s))
```

Q002 | Stable list partition by multiple rules

Source Implementation:

```
def stable_partition(values):




```

Verified Output:

```
[-2, -5, 0, 0, 4, 7, 3]
```

Q003 | Circular window statistics

Source Implementation:

```
def circular_window_stats(values, k):



                conflicts = sorted(list(del_s & (read_s | write_s)))
```

Verified Output:

```
[4.0, 6.0, 5.33, 4.67]
```

Q004 | Tuple record normalization

Source Implementation:

```
def normalize_records(records):




```

Verified Output:

```
((4, 'Mina', 'HR', 48000), (2, 'Ravi', 'HR', 42000), (3, 'Asha', 'IT', 56000))
```

Q005 | Set-based access audit

Source Implementation:

```
def access_audit(read_s, write_s, del_s, all_u):



```

Verified Output:

```
conflicts=[3, 7]; all_three=[7]; none=[6, 8]
```

Q006 | Dictionary inversion with collisions

```
                if not s: return None
```

Source Implementation:

```
def invert_dict(d):
```

```
                                        for r_idx, row in enumerate(matrix):
```

Verified Output:

```
{2: ['red', 'green'], 1: ['blue', 'black']}
```

Q007 | Nested inventory reconciliation

Source Implementation:

```
def reconcile_inventory(a, b):
```

```
                            row_map = defaultdict(list)
```

Verified Output:

```
{'pen': {'Mumbai': 7, 'Pune': 15, 'Nashik': 4}, 'bag': {'Nashik': 2}, 'book': {'Pune': 3}}
```

Q008 | String canonicalization

Source Implementation:

```
def canonicalize(text):
```

Verified Output:

```
datascience is fun
```

Q009 | Immutable string run analysis

Source Implementation:

```
def longest_run(s):
                counts = defaultdict(int)
```

Verified Output:

```
('c', 4, 5)
```

Q010 | Nested collection frequency

Source Implementation:

```
def nested_frequencies(matrix):
```

Verified Output:

```
{1: (3, [0, 2]), 2: (2, [0, 1]), 3: (3, [1, 2])}
```

Q011 | Dictionary key dependency chain

Source Implementation:

```
def resolve_deps(deps):
```

```
                m, n = len(s1), len(s2)
```

Verified Output:

```
(True, ['lint', 'test', 'build'])
```

Q012 | Two-list stable multiset difference

Source Implementation:

```
def stable_multiset_diff(a, b):
```

Verified Output:

```
[3, 2, 4, 1]
```

Q013 | String edit-distance threshold

Source Implementation:

```
def is_one_edit_distance(s1, s2):
```

Verified Output:

```
True
```

```
                    lean = (y % 4 == 0 and y % 100 != 0) or (y % 400 == 0)
```

Q014 | Boolean expression evaluator

Source Implementation:

```
def eval_boolean_expr(expr, values):
```

```
                if y < 1 or m < 1 or m > 12 or d < 1: return "Invalid date."
```

Verified Output:

```
True
```

Q015 | Progressive tax calculation

Source Implementation:

```
def progressive_tax(income):
```

Verified Output:

```
Tax: 132500.00; Effective rate: 9.81%
```

Q016 | Date validity without date libraries

Source Implementation:

```
def validate_date(d, m, y):
```

Verified Output:

```
Invalid date.
```

Q017 | Sequence stopping with dual conditions

Source Implementation:

```
def sequence_stopping(values):
    s = str(n)
    pairs = []
```

Verified Output:

```
processed=[8, 12, 0, 4, 0, 0, 0]; count=7; sum=24; reason='three consecutive zeros'
```

Q018 | Prime gap detector

Source Implementation:

```
def prime_gap(L, R):
    seen = set()
```

Verified Output:

```
(23, 29) gap=6
```

Q019 | Digit pattern classification

Source Implementation:

```
def classify_digits(n):
```

Verified Output:

```
Strictly increasing
```

Q020 | Bounded nested-loop pair search

Source Implementation:

```
def bounded_pair_search(values, target):
```

Verified Output:

```
[(0, 3), (4, 5)]
```

```
                                        if C == 1: return [matrix[r][0] for r in range(R)]
```

Q021 | Alternating harmonic accumulator

Source Implementation:

```
def alternating_harmonic(n):
```

Verified Output:

```
0.783333
```

Q022 | Run-length threshold detector

Source Implementation:

```
def run_length_threshold(values, k):
                                    if R == 1: return matrix[0]
```

```
                res = [
```

Verified Output:

```
(2, 2, 3)
```

```
                            R, C = len(matrix), len(matrix[0])
```

Q023 | Perfect-number range with divisor optimization

Source Implementation:

```
def perfect_numbers(L, R):
```

Verified Output:

```
[6, 28, 496]
```

```
                if not matrix or not matrix[0]: return []
```

Q024 | Multi-rule number filter

Source Implementation:

```
def multi_rule_filter(n):
```

Verified Output:

```
values=[6, 20, 24, 28] count=4
```

Q025 | Matrix boundary traversal

Source Implementation:

```
def matrix_boundary(matrix):
```

Verified Output:

```
[1, 2, 3, 4, 8, 12, 11, 10, 9, 5]
```

Q026 | Automorphic numbers in a range

Source Implementation:

```
def automorphic_range(L, R):
```

```
                        a, b, res = 0, 1, []
```

Verified Output:

```
[1, 5, 6, 25, 76, 376]
```

Q027 | Armstrong numbers for arbitrary digit length

Source Implementation:

```
def armstrong_range(L, R):
```

Verified Output:

```
[1, 2, 3, 4, 5, 6, 7, 8, 9, 153, 370, 371, 407, 1634, 8208, 9474]
```

```
                is_p = lambda x: x > 1 and all(x % i != 0 for i in range(2, int(x**0.5) + 1))
```

Q028 | Prime factorization with exponents

Source Implementation:

```
def prime_factorization(n):
```

Verified Output:

```
[(2, 2), (3, 3), (7, 1)]
```

Q029 | Fibonacci range intersection

Source Implementation:

```
def fib_prime_odd_digitsum(N):
```

Verified Output:

```
[3, 5, 89]
```

```
    if n == 0: return "0"
```

Q030 | GCD and LCM across a list

```
    l = len(s)
```

Source Implementation:

```
def list_gcd_lcm(values):
```

Verified Output:

```
GCD=12; LCM=360
```

Q031 | Digit permutation divisibility

Source Implementation:

```
def digit_permutation_divisibility(n):
```

Verified Output:

```
False
```

Q032 | Binary-to-decimal validation

Source Implementation:

```
def binary_to_decimal(s):
    s = str(n)
```

Verified Output:

```
45
```

Q033 | Decimal-to-base conversion

Source Implementation:

```
def decimal_to_base(n, b):
```

Verified Output:

```
FF
```

Q034 | Nearest palindrome number

Source Implementation:

```
def nearest_palindrome(n):
```

Verified Output:

```
121
```

Q035 | Concentric alphabet square

Source Implementation:

```
def concentric_alphabet_square(N):
```

Verified Output:

```
#*  *#
lines = []
```

Q036 | Hollow mirrored number diamond

Source Implementation:

```
def hollow_mirrored_diamond(N):
```

Verified Output:

```
*
*
```

Q037 | Alternating hollow butterfly

Source Implementation:

```
def alternating_hollow_butterfly(N):
```

Verified Output:

```
    grid = []
```

Q038 | Spiral matrix with alternating rings

Source Implementation:

```
def spiral_alternating_rings(n):
```

Verified Output:

```
                for r in range(1, N + 1):
```

ABC

Q039 | Pascal triangle with row sums

Source Implementation:

```
def pascal_with_sums(N):
```

```
                        *******
```

Verified Output:

```
                    3   3
    lines = []
```

Q040 | Alphabet snake matrix

Source Implementation:

```
def alphabet_snake_matrix(n):
```

```
            * *
```

Verified Output:

Q041 | Composite hollow pattern

Source Implementation:

```
def composite_hollow_pattern(N):
```

Verified Output:

```
    if not data: return []
```

Q042 | Grouped comprehension transformation

Source Implementation:

```
def grouped_word_comprehension(sentences, k):
                                                    if predicate(curr): res.append(curr)
```

Verified Output:

```
{0: ['powerful', 'python'], 1: ['powerful', 'python', 'tools']}
```

Q043 | Matrix comprehension filter

Source Implementation:

```
def matrix_comprehension_filter(matrix):          for t in transforms: curr = t(curr)
```

Verified Output:

```
    memo = {}
[(0, 0, 2)]
```

```
                                          curr = dict(r)
```

Q044 | Dictionary comprehension with collision policy

Source Implementation:

```
def dict_comprehension_sum(pairs):
```

Verified Output:

```
{'A': 9, 'B': 5, 'C': 1}
```

Q045 | Recursive nested-list flattening

Source Implementation:

```
def recursive_flatten(data):    for r in records:
```

Verified Output:

```
[1, 2, 3, 4, 5, 6]
```

Q046 | Recursive path count in a grid

Source Implementation:

```
def grid_paths_count(m, n, blocked):    res = []
```

Verified Output:

```
2
```

Q047 | Higher-order pipeline

Source Implementation:

```
def pipeline(records, transforms, predicate):
```

Verified Output:

```
[{'name': 'B', 'salary': 71500}]
```

Q048 | Lambda-based multi-key ranking

Source Implementation:

```
def rank_employees(employees):
    for item in iterable:
```

Verified Output:

```
[('Ravi', 'HR', 50000), ('Aman', 'HR', 45000), ('Mira', 'IT', 60000), ('Zoya', 'IT', 60000)]
```

Q049 | Enumerate and zip reconciliation

Source Implementation:

```
def stock_reconciliation(ids, a, b, threshold):
```

Verified Output:

```
[(0, 'P1', 7), (2, 'P3', 7)]
```

Q050 | Scope-safe counter factory

Source Implementation:

```
def make_counter(start=0):
    chunk = []
```

Verified Output:

```
c1=15; c2=5
```

Q051 | Decorator with execution audit

Source Implementation:

```
def audit(func):
```

Verified Output:

```
return=12, audit={'args': (3,), 'kwargs': {'y': 4}, 'result': 12, 'duration_ms': 0.0035}
```

Q052 | Generator for chunked records

Source Implementation:

```
def chunked_generator(iterable, k):
```

Verified Output:

```
[[1, 2], [3, 4], [5]]
```

Q053 | Iterator with look-ahead

Source Implementation:

```
def look_ahead_iterator(seq):
    tokens = re.findall(r'\(|\)|\d+|\+|\-|\*', expr)
```

Verified Output:

```
[('a', 'b'), ('b', 'c'), ('c', None)]
    class RangeError(ParseError): pass
```

Q054 | Memoization decorator

Source Implementation:

```
def memoize(func):
```

Verified Output:

```
result=55, hits=9, misses=11
class EmptyInputError(ParseError): pass
```

Q055 | Recursive expression evaluator

Source Implementation:

```
def eval_parenthesized(expr):
```

Verified Output:

```
Execution Error: list index out of range
```

Q056 | Validated numeric input parser

Source Implementation:

```
class ParseError(Exception): pass
```

Verified Output:

```
RangeError: value must be between 0 and 10
```

Q057 | Retry-controlled transaction

Source Implementation:

```
def retry_transaction(simulated_outcomes):
```

```
    inv = {k.strip(): int(v.strip()) for k, v in inv.items()}
```

```
# models.py: User, BillingRecord
```

```
    return f"popped='{popped}'; remaining={records}"
```

Verified Output:

```
Success on attempt 3; cleanup executed
```

Q058 | Custom exception hierarchy

Source Implementation:

```
class InsufficientFundsError(Exception): pass
```

```
    popped = records.pop()
```

Verified Output:

```
InsufficientFundsError, balances remain A: 500, B: 100
```

```
    inv = dict(l.split(',') for l in content.split('\n') if l.strip())
```

```
    if not records: raise EmptyStackError("Empty stack")
```

Q059 | Package import design

Source Implementation:

```
def area_circle(r): return round(math.pi * r * r, 4)
```

Verified Output:

```
area_circle(2) -> 12.5664; area_rectangle(3, 4) -> 12
```

```
book, 8
```

```
def file_stack_pop(records):
```

Q060 | Module dependency and circular import repair

Source Implementation:

```
# Models module holding shared state/contracts resolves circular import:
```

Verified Output:

```
Both modules import shared models.py; imports complete without circular initialization
```

Q061 | File-backed inventory update

Source Implementation:

```
def safe_update_inventory(content, item, qty_delta):
```

Verified Output:

```
pen, 10
```

Q062 | File record stack operations

Source Implementation:

```
class EmptyStackError(Exception): pass
```

Verified Output:

```
popped='T2'; remaining=['T1']
```

```
                                st, en = max(s1, s2), min(e1, e2)
```

Q063 | File merge with duplicate resolution

Source Implementation:

```
def merge_csv_streams(lines1, lines2):
```

Verified Output:

```
B,15,b
```

```
                best, md = None, -1.0
```

Q064 | Regex log extraction

```
                                s2, e2 = datetime.date.fromisoformat(B[0]), datetime.date.fromisoformat(B[1])
```

Source Implementation:

```
def parse_logs(logs):
```

Verified Output:

```
{'auth': 1, 'db': 1}
```

Q065 | Regex overlapping token detection

Source Implementation:

```
def find_overlapping_tokens(text):
```

```
                s1, e1 = datetime.date.fromisoformat(A[0]), datetime.date.fromisoformat(A[1])
```

Verified Output:

```
[(0, 'a1b'), (3, 'a2c'), (6, 'a3d')]
```

Q066 | Math module numerical stability

Source Implementation:

```
def furthest_point(points):
```

Verified Output:

```
((6, 8), 10.0)
```

Q067 | Date interval overlap

Source Implementation:

```
def date_interval_overlap(A, B):
```

Verified Output:

```
Overlap=True; days=3
```

```
                                errs = []
```

Q068 | Business-day deadline calculator

Source Implementation:

```
def calculate_business_deadline(start_str, working_days, holidays):

                """Compute total sum of elements following PEP 8 conventions."""
```

Verified Output:

```
2026-11-05
```

Q069 | Environment reproducibility report

Source Implementation:

```
                def load_config(cfg):
def env_report(packages, installed):
                total = 0
```

Verified Output:

```
numpy: installed 1.26.4, meets minimum: True; requests: not installed
```

Q070 | Dependency conflict investigation

Source Implementation:

```
def check_dependency_conflicts(reqs, installed):
```

Verified Output:

```
Incompatible numpy 1.23; command: python -m pip install 'numpy>=1.24'
```

Q071 | PEP 8 refactoring audit

Source Implementation:

```
def calc_total(values):
```

Verified Output:

```
def calc_total(values):
```

Q072 | Safe configuration loader

Source Implementation:

```
class ConfigurationError(Exception): pass
```

Verified Output:

```
ConfigurationError: PORT must be an integer; missing required key: SECRET_KEY
```

Q073 | Bank account with invariant protection

Source Implementation:

```
class BankAccount:
```

```
def charge(self, amount): pass
```

Verified Output:

```
A.balance=300; B.balance=300; transfer recorded in both histories
```

Q074 | Employee payroll polymorphism

Source Implementation:

```
class Employee:
```

```
class B(A): pass
```

```
@abstractmethod
```

Verified Output:

```
[('A', 50000), ('B', 8000), ('C', 45000)]
```

Q075 | Class-level ID allocation

Source Implementation:

```
class SequentialIDEntity:
```

```
class PaymentGateway(ABC):
```

Verified Output:

```
IDs: E001, E002, E003; count=3
```

Q076 | Inheritance and method resolution

Source Implementation:

```
class A: pass
```

Verified Output:

```
MRO: DBCA -> object
```

Q077 | Abstract payment gateway

Source Implementation:

```
from abc import ABC, abstractmethod
```

Verified Output:

```
Payment of 1200 processed through CardGateway
```

Q078 | Composition-based order system

Source Implementation:

```
def order_composition_totals(items, tax_pct, disc_pct):
```

Verified Output:

```
Subtotal=660; Tax=33; Discount=66; Total=627
```

```
class ExportJSON:
    def __init__(self, path): self.path = path
```

Q079 | Encapsulation with validated properties

Source Implementation:

```
class Student:
```

Verified Output:

```
marks remain 85; grade='A'
```

Q080 | Operator overloading for vectors

```
    def export_csv(self): return "CSV representation"
```

Source Implementation:

```
class Vector2D:
```

Verified Output:

```
v1+v2 -> Vector2D(4, 6); v2-v1 -> Vector2D(2, 2)
```

Q081 | Context manager for resource tracking

Source Implementation:

```
class ManagedResource:
```

Verified Output:

```
Resource closed; ValueError propagated
```

Q082 | Multiple inheritance capability mixins

Source Implementation:

```
class ExportCSV:
```

Verified Output:

```
CSV and JSON representations generated from identical records
```

Q083 | Descriptor for validated fields

Source Implementation:

```
class PositiveNumeric:
    def __init__(self, pct): self.pct = pct
```

Verified Output:

```
price=99.5; weight assignment raises ValueError
```

Q084 | Custom iterable class

Source Implementation:

```
class Playlist:
```

Verified Output:

```
['A', 'B', 'C']; length=3; membership=True
```

Q085 | Factory method with validation

Source Implementation:

```
class User:
```

Verified Output:

```
User(id='U7', name='Asha', email='asha@example.com', age=22)
```

Q086 | Strategy pattern for discounts

Source Implementation:

```
class PercentageDiscount:
```

Verified Output:

```
850
```

Q087 | Observer pattern for stock alerts

Source Implementation:

```
class StockSubject:
```

Verified Output:

```
Registered observers notified once: 'Low stock: 4'
```

Q088 | OOP mini-system: library circulation

Source Implementation:

```
def library_circulation_demo():
```

Verified Output:

```
Book returned; overdue_days=0; fine=0
```

```
                shape = (len(arr) - k + 1, k)
```

Q089 | Masked normalization by column

Source Implementation:

```
def masked_normalization(arr):
```

Verified Output:

```
[[0.0, nan, 0.0], [0.5, nan, 0.0], [1.0, nan, 0.0]]
```

Q090 | Broadcasted row-column distance matrix

Source Implementation:

```
def broadcasted_distance_matrix(A, B):

                res = arr.copy().astype(float)
```

Verified Output:

```
[[1.0, 2.8284, 2.0], [1.0, 1.4142, 1.4142]]
```

Q091 | Stable row ranking

```
                return A @ B
```

Source Implementation:

```
def stable_row_ranking(arr):
```

Verified Output:

```
indices=[1, 2, 0, 3]; sorted_rows=[[1, 3], [4, 0], [2, 1], [0, 0]]
```

Q092 | Sliding-window array statistics

Source Implementation:

```
                raise ValueError(f"Incompatible shapes: {A.shape} and {B.shape}")
def sliding_window_stats(arr, k):
```

Verified Output:

```
means=[2.0, 3.0, 4.0]; std=[0.8165, 0.8165, 0.8165]
```

Q093 | Replace outliers with column medians

Source Implementation:

```
def replace_outliers_mad(arr):
                if A.shape[1] != B.shape[0]:
```

Verified Output:

```
[[10.0, 1.0], [11.0, 2.0], [12.0, 3.0], [11.5, 4.0]]
```

Q094 | Matrix multiplication compatibility

Source Implementation:

```
def validated_matmul(A, B):
```

Verified Output:

```
result.shape=(2, 4)
```

```
                res = []
```

Q095 | Group-wise aggregation with unique labels

Source Implementation:

```
def group_wise_aggregation(values, labels):
```

Verified Output:

```
                    norm_B = np.linalg.norm(B, axis=1, keepdims=True)
[('A', 2, 30.0, 40), ('B', 3, 30.0, 50)]
```

Q096 | Pairwise correlation matrix

Source Implementation:

```
def safe_pairwise_correlation(arr):
```

Verified Output:

```
corr between columns 0 and 1=1.0; correlations involving constant column=nan
```

```
            cm = np.zeros((num_classes, num_classes), dtype=int)
```

Q097 | Diagonal extraction and reconstruction

Source Implementation:

```
def diagonal_reconstruction(arr):
                    norm_A = np.linalg.norm(A, axis=1, keepdims=True)
```

Verified Output:

```
[[1, 0, 3], [0, 5, 0], [7, 0, 9]]
```

Q098 | Top-k values per row

Source Implementation:

```
def top_k_per_row(arr, k):
```

```
            dot = A @ B.T
```

Verified Output:

```
[[1, 3], [2, 0]]
```

Q099 | Vectorized confusion matrix

Source Implementation:

```
def confusion_matrix(y_true, y_pred, num_classes):
```

Verified Output:

```
[[2, 0, 0], [0, 1, 1], [0, 0, 1]]
```

Q100 | Memory-aware pairwise similarity

Source Implementation:

```
def vectorized_cosine_similarity(A, B):
```

Verified Output:

```
[[0.7071, 1.0], [0.7071, 0.0]]
```